Lec09：开发规范

Software Development Specification

课程编号：22WHSE22006（2023版培养计划）
授课对象：23 级软件工程、服务科学与工程专业
主讲教师：辛国栋
Email：xinguodong@163.com



QQ 群：软件构造2025
群　号：311528424

如何写出高质量的代码？ – 编写高质量代码的方法论

面向对象

- 封装、抽象、继承、多态
- 面向对象编程 vs 面向过程编程
- 面向对象分析、设计、编程
- 接口 vs 抽象类
- 基于接口而非实现编程
- 多用组合，少用继承
- 贫血模型和充血模型

基础

设计原则

- SRP单一职责
- OCP开闭原则
- LSP里氏代换原则
- ISP接口隔离原则
- DIP依赖倒置原则

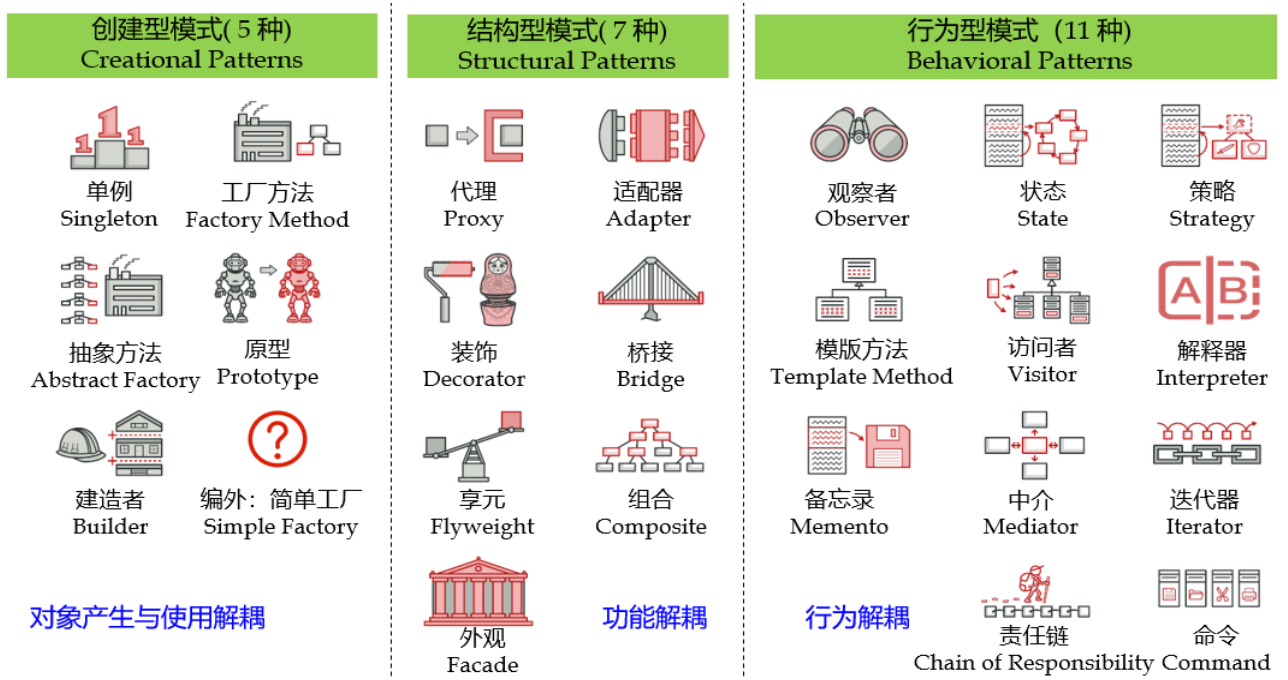
SOLID

- DRY、KISS、YAGNI
- LOD法则

内功

设计模式

招式



开发规范：编码规范、异常日志规范等

规矩

重构：四要素、单元测试与代码的可测试性等

修炼

课程内容

① 面向对象(课堂 6学时)
第 1-3 章

② 设计原则(课堂 4 学时)
第 4-5 章，实验 1（实验 4 学时）

③ 设计模式（课堂 14 学时）
第 6-8 章，实验 2-4（实验 12 学时 ）

④ 开发规范（课堂4学时）第9章

⑤ 重构（课堂4学时）第10章

	序号	方式	授课内容	学时
基础	1	讲课	第 1 章 类、类间关系的UML	2
	2	讲课	第 2 章 面向对象特征及其解决的问题	2
	3	习题	第 3 章 面向对象编程习题课	2
内功	4	讲课	第 4 章 SOLID	2
	5	讲课	第 5 章 其他原则	2
	6	实验	实验1：开发基础（注解、反射）	4
招式	7	讲课	第 6 章 创建型设计模式	6
	8	实验	实验2：Bean容器构建（IOC）	4
	9	讲课	第 7 章 结构型设计模式	4
	10	实验	实验3：对象的自动装配（DI）	4
	11	讲课	第 8 章 行为型设计模式	4
	12	实验	实验4：重构	4
规矩	13	讲课	第 9 章 代码风格	4
修炼	14	讲课	第10章 重构	4

课堂 32 学时/综合实验 16 学时。第10-17 周，11月04日~12月25日

引言：可读性如何落地？- 开发规范

```
                #include "stdafx.h"
            #include                "stdio.h"
          void test();
                   int _tmain
                   (int argc,
                      _TCHAR*           argv[])
                          { test(); return
                                      0; }

                                  char C[25][40];void d(int x,int y)
                {C[x][y]=C[x][y+1]=32;}int f(int x){return (int)x*x*.08;}
                void test(){int i,j;                                    char
s[5]="TEST";
           for(i=0;i<25;i++)
          for(j=0;j<40;j++)
         C[i][j]=s[(i+j)%4];
        for(i=1;i<=7;i++)
        {d(18-i,12);
        C[20-f(i)][i+19]=
         C[20-f(i)][20-i]=32;
          }d(10,13);d(9,13);
           d(8,14);d(7,15);
              d(6,16);d(5,18);d(5,20);                          d(5,22);d
(5,26);
                  d(6,23);d(6,25);d(7,25);for(i=0;i<25;i++,printf("\n"))
                                    for(j=0;j<40;printf("%c",C[i]
[j++]));}
```

```
161     ttParams.m_bVislManagerTheme = TRUE;
162     theApp.GetTooltipManager()->SetTooltipParams(AFX_TOOLTIP_TYPE_ALL,
163         RUNTIME_CLASS(CMFCToolTipCtrl), &ttParams);
164     ////////////////////////////////////////////////////////////////////
165     //PS:我也不知道这为啥这么写，反正去掉这块程序就好使了
166     //
167     //CShadoWinRDPMFCDlg dlg;
168     //dlg.DoModal();
169
170     ////////////////////////////////////////////////////////////////////
171     // 若要创建主窗口，此代码将创建新的框架窗口
172     // 对象，然后将其设置为应用程序的主窗口对象
173
174     CMainFrame* pFrame = new CMainFrame;
175     if (!pFrame)
176         return FALSE;
```

```
function register()
{
    if (!empty($_POST)) {
        $msg = '';
        if ($_POST['user_name']) {
            if ($_POST['user_password_new']) {
                if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {
                    if (strlen($_POST['user_password_new']) > 5) {
                        if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {
                            if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {
                                $user = read_user($_POST['user_name']);
                                if (!isset($user['user_name'])) {
                                    if ($_POST['user_email']) {
                                        if (strlen($_POST['user_email']) < 65) {
                                            if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {
                                                create_user();
                                                $_SESSION['msg'] = 'You are now registered so please login';
                                                header('Location: ' . $_SERVER['PHP_SELF']);
                                                exit();
                                            } else $msg = 'You must provide a valid email address';
                                        } else $msg = 'Email must be less than 64 characters';
                                    } else $msg = 'Email cannot be empty';
                                } else $msg = 'Username already exists';
                            } else $msg = 'Username must be only a-z, A-Z, 0-9';
                        } else $msg = 'Username must be between 2 and 64 characters';
                    } else $msg = 'Password must be at least 6 characters';
                } else $msg = 'Passwords do not match';
            } else $msg = 'Empty Password';
        } else $msg = 'Empty Username';
        $_SESSION['msg'] = $msg;
    }
    return register_form();
}
```





引言：可读性如何落地？- 开发规范

```
$sql = "select * from `{bbs_forums}` {$_sql} order by `order` desc";
$result = $mysql->db_fetch_arrays($sql);
$_result = "";
$lgtid = isset($data['lgtid'])?$data['lgtid']:"";
if (count($result)>0){
        $i=0;
        if (!isset($code) || $code=="") {
                foreach($result as $key => $value){
                        if ($value['pid']==0 && $value['id']!=$lgtid){
                                $_result[$i] = $value;
                                $_result[$i]['aname'] = "<b>".$value['name']."</b>";
                                $_result[$i]['porder'] = "";
                                $_result[$i]['norder'] = 1;
                                $i++;
                                foreach($result as $_key => $_value){
                                        if ($_value['pid']==$value['id'] && $_value['id']!=$lgtid){
                                                $_result[$i] = $_value;
                                                $_result[$i]['aname'] = "-".$_value['name'];
                                                $_result[$i]['porder'] = "|——";
                                                $_result[$i]['norder'] = 2;
                                                $i++;
                                                foreach($result as $__key => $__value){
                                                        if ($__value['pid']==$_value['id'] && $__value
                                                                $_result[$i] = $__value;
                                                                $_result[$i]['aname'] = "--".$__value[
                                                                $_result[$i]['porder'] = "|————";
                                                                $_result[$i]['norder'] = 3;
                                                                $i++;
                                                                foreach($result as $___key => $___valu
                                                                        if ($___value['pid']==$__value
                                                                                $_result[$i] = $___val
                                                                                $_result[$i]['aname']
                                                                                $_result[$i]['porder']
                                                                                $i++;
                                                                        }
                                                                }
                                                        }
                                                }
                                        }
                                }
                        }
                }
```

神鬼难测

引言：可读性如何落地？- 开发规范

Any fool can write code that a computer can understand. Good programmers write code that humans can understand

Martin Fowler

代码可读性：代码是否符合编码规范、命名是否达意、注释是否详尽、函数是否长短合适、模块划分是否清晰、是否符合高内聚低耦合。

code review 是一个很好的测验代码可读性的手段。若同事可以轻松地读懂你写的代码，说明你的代码可读性很好；如果同事在读你的代码时，有很多疑问，那就说明你的代码可读性有待提高

引言：可读性如何落地？- 开发规范



一、编程规约 1
(一) 命名风格 1
(二) 常量定义 4
(三) 代码格式 5
(四) OOP 规约 7
(五) 日期时间 11
(六) 集合处理 12
(七) 并发处理 17
(八) 控制语句 20
(九) 注释规约 24
(十) 前后端规约 25
(十一) 其他 27
二、异常日志 29
(一) 错误码 29
(二) 异常处理 30
(三) 日志规约 32
三、单元测试 35
四、安全规约 37
五、MySQL 数据库 38
(一) 建表规约 38
(二) 索引规约 39
(三) SQL 语句 41
(四) ORM 映射 42
六、工程结构 44
(一) 应用分层 44
(二) 二方库依赖 45
(三) 服务器 46
七、设计规约 48

1. 命名风格
Naming Style

1.0 命名能力体现了一个程序员的基本编程素养

- 项目名、模块名、包名、对外暴露的接口，类名、函数名、变量名、参数名，都需要 “起名字” 。命名的好坏，对于代码的可读性来说非常重要。
- 命名能力体现了一个程序员的基本编程素养

```
int  i;
int  j;
int lili;
float yunyun;
```



```
boolean fFileExist; //文件是否存在的 boolean值，匈牙利命名法。
int studentNo; //整形变量-学号，驼峰命名法。
void print_employee_paychecks() //打印支票，下划线表示法
```



- 有同学会有异议：命名不就是不能用 abcxyz 命名，名字要有意义，命名要可读，可搜索等，这有什么好讲的?

1.1 缺乏业务含义的命名：①不精准的命名

请快速阅读以下代码，分析这段代码的作用。

```
public void processChapter(long chapterId) {
    Chapter chapter = this.repository.findByChapterId(chapterId);
    if (chapter == null) {
        throw new IllegalArgumentException("Unknown chapter [" + chapterId + "]");
    }
    chapter.setTranslationState(TranslationState.TRANSLATING);
    this.repository.save(chapter);
}
```

这段代码做的就是把一个章节的翻译状态改成“翻译中” 。

问题来了，为什么你需要阅读这段代码的细节，才能知道这段代码是做什么的?

说明方法命名不够精确。必须通过读代码才能明白方法的具体作用

1.1 缺乏业务含义的命名：①不精准的命名

函数 processChapter 确实是在处理章节，但是，这个名字太过宽泛。

```
public void processChapter(long chapterId) {
    Chapter chapter = this.repository.findByChapterId(chapterId);
    if (chapter == null) {
        throw new IllegalArgumentException("Unknown chapter [" + chapterId + "]");
    }
    chapter.setTranslationState(TranslationState.TRANSLATING);
    this.repository.save(chapter);
}
```

- ✓ 如果说“将章节的翻译状态改成翻译中”叫做处理章节，那么“将章节的翻译状态改成翻译完”是不是也叫处理章节呢？“修改章节内容”是不是也叫处理章节呢？

- ✓ 换句话说，如果各种场景都能够叫处理章节，那么处理章节就是一个过于宽泛的名字，不能说命名错误，但不精准。

命名过于宽泛，不能精准描述，这是很多代码在命名上存在的严重问题，也是代码难以理解的根源所在。

1.1 缺乏业务含义的命名：①不精准的命名

回到前面那段代码上，如果它不叫“处理章节”，那应该叫什么呢?

命名要能够描述出这段代码在做的事情。

方案1：修改为：changeChapterToTranlsating

分析：不算是一个好名字，因为它更多的是在描述这段代码在做的细节。

之所以要将一段代码封装起来，一个重要的原因就是，我们不想知道那么多的细节。如果把细节平铺开来，那本质上和直接阅读代码细节差别并不大。

一个好的名字应该描述意图，而非细节



方案2：修改为：startTranslation

分析：为什么要把翻译状态修改成翻译中，这一定是有原因的，也就是意图。具体到这里的业务，我们把翻译状态修改成翻译中，是因为我们在这里开启了一个翻译的过程。所以，这段函数应该命名startTranslation。



1.1 缺乏业务含义的命名：①不精准的命名

```
public void processChapter(long chapterId) {
    Chapter chapter = this.repository.findByChapterId(chapterId);
    if (chapter == null) {
        throw new IllegalArgumentException("Unknown chapter [" + chapterId + "]");
    }
    chapter.setTranslationState(TranslationState.TRANSLATING);
    this.repository.save(chapter);
}
```





```
public void startTranslation(long chapterId) {
    Chapter chapter = this.repository.findByChapterId(chapterId);
    if (chapter == null) {
        throw new IllegalArgumentException("Unknown chapter [" + chapterId + "]");
    }
    chapter.setTranslationState(TranslationState.TRANSLATING);
    this.repository.save(chapter);
}
```



函数名修改后，很容易通过函数名就 get 到代码的意图，提高了易读性，也符合封装的本意

1.1 缺乏业务含义的命名：②用技术术语命名

```
List<Book> bookList = service.getBooks();
```



这段代码隐藏另外一个典型得不能再典型的问题：用技术术语命名。

这个 bookList 变量之所以叫 bookList，原因就是它声明的类型是 List。这种命名在代码中几乎是随处可见的，比如 xxxMap、xxxSet。这种命名却会带来很多问题，因为它是一种基于实现细节的命名方式

假设现在需要的是一个不重复的作品集合，即需要把变量的类型从 List 改成 Set。变量类型你一定会改，但变量名你会改吗?
这还真不一定，一旦出现遗忘，就会出现一个奇特的现象，一个叫 bookList 的变量，它的类型是一个 Set。这样，一个新的混淆就此产生了。

需要一个更面向意图的名字。这段代码里真正要表达的是拿到了一堆书，所以，这个名字可以命名成 books

```
List<Book> books = service.getBooks();
```



1.1 缺乏业务含义的命名：②用技术术语命名

```
public Book getByIsbn(String isbn) {
    Book cachedBook = redisBookStore.get(isbn);
    if (cachedBook != null) {
        return cachedBook;
    }
    Book book = doGetByIsbn(isbn);
    redisBookStore.put(isbn, book);
    return book;
}
```

在实际的代码中，技术名词的出现，往往就代表着它缺乏抽象，未能从业务角度命名：

```
public Book getByIsbn(String isbn) {
    Book cachedBook = cache.get(isbn);
    if (cachedBook != null) {
        return cachedBook;
    }
    Book book = doGetByIsbn(isbn);
    cache.put(isbn, book);
    return book;
}
```

比上面的代码好一些。但缓存这个概念其实也是一个技术术语，它也不应该出现在业务代码中

```
@Cacheable("books")
public Book getByIsbn(String isbn) {
    //...
}
```

做得比较好的是 Spring。使用 Spring 框架时，如果需要缓存，我们通常是加上一个 Annotation（注解）：

1.1 缺乏业务含义的命名：结论

无论是不精准的命名还是使用技术术语命名，体现了同一个问题：对业务理解不到位。例如：

```
public void approveChapter(long chapterId, long userId) {
    //...
}
```



函数的意图是确认章节内容审核通过。chapterId 是审核章节的 ID，这个没问题，但 userId 是什么呢？

因为这里需要记录一下审核人的信息，这个 userId 就是审核人的 userId

userId 并不是一个好的命名，因为它还需要更多的解释，更好的命名是 reviewerUserId，之所以起这个名字，因为这个用户在这个场景下扮演的角色是审核人（Reviewer）。

```
public void approveChapter(long chapterId, long reviewerUserId) {
    //...
}
```



总结：好的命名，是体现业务含义的命名

1.2 乱用英语：①违反英语语法规则的命名

```
public void completedTranslate(final List<ChapterId> chapterIds) {
    List<Chapter> chapters = repository.findByChapterIdIn(chapterIds);
    chapters.forEach(Chapter::completedTranslate);
    repository.saveAll(chapters);
}
```

这段代码写得还不错，它要做的是将一些章节的信息标记为翻译完成，似乎函数名也能反映这个意思，但仔细一看你就会发现问题

因为 completedTranslate 并不是一个正常的英语函数名。作者想表达的是“完成翻译”，因为是已经翻译完了，所以，他用了完成时的 completed，函数名就成了 completedTranslate。作者已经很用心，但名字还是起错了。

常见的命名规则是：类名是一个名词，表示一个对象，而方法名则是一个动词，或者是动宾短语，表示一个动作

1.2 乱用英语：①违反英语语法规则的命名

以此为标准衡量，completedTranslate 并不是一个有效的动宾结构，而应该是：completeTranslation

```
public void completeTranslate(final List<ChapterId> chapterIds) {
    List<Chapter> chapters = repository.findByChapterIdIn(chapterIds);
    chapters.forEach(Chapter::completedTranslate);
    repository.saveAll(chapters);
}
```

这种坏味道在代码中时常可以见到，比如，一个函数名是 reTranslation，其表达的意图是重新翻译，但作为函数名，它应该是一个动词，所以，正确的命名应该是 reTranslate。

1.2 乱用英语：②不准确的英语词汇

例如：实现一个章节审核的功能，一个同事先定义出了审核的状态

```
public enum ChapterAuditStatus {
    PENDING,
    APPROVED,
    REJECTED;
}
```

audit 会有更官方的味道，更合适的翻译应该是审计，而 review 则有更多核查的意思，二者相比，review 更适合这里的场景。于是，章节的审核状态统一使用 review

```
public enum ChapterReviewStatus {
    PENDING,
    APPROVED,
    REJECTED;
}
```



1.2 乱用英语：③英语单词的拼写错误

```
public class QuerySort {
    private final SortBy sortBy;
    private final SortFiled sortFiled;
    //...
}
```

sortFiled 是啥？排序文件吗？为啥用的还是过去式？归档？

实际上表示的是“排序的字段”，field 写错成 filed，造成理解的困扰

1.3 阿里命名风格介绍

- 1.【强制】代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。

  反例：`_name / __name / $name / name_ / name$ / name__`

- 2.【强制】所有编程相关的命名严禁使用拼音与英文混合的方式，更不允许直接使用中文。

  说明：正确的英文拼写和语法可以让阅读者易于理解，避免歧义。注意，纯拼音命名方式更要避免采用

  正例：`ali / alibaba / taobao / cainiao/ aliyun/ youku / hangzhou` 等国际通用的名称，可视同英文。

  反例：`DaZhePromotion [打折] / getPingfenByName() [评分] / String fw[福娃] / int 某变量 = 3`

- 3.【强制】代码和注释中都要避免使用任何语言的种族歧视性词语。

  正例：`日本人 / 印度人 / secondary`

  反例：`RIBENGUIZI / Asan / slave`

1.3 阿里命名风格介绍

- 4.【强制】类名使用UpperCamelCase风格，但以下情形例外：DO / BO / DTO / VO / AO / PO / UID等。

```
正例：ForceCode / UserDO / HtmlDTO / XmlService / TcpUdpDeal / TaPromotion
反例：forcecode / UserDo / HTMLDto / XMLService / TCPUDPDeal / TAPromotion
```

- 5.【强制】方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格。

```
正例：  localValue / getHttpMessage() / inputUserId
```

- 6.【强制】常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长

```
正例：MAX_STOCK_COUNT / CACHE_EXPIRED_TIME
反例：MAX_COUNT / EXPIRED_TIME
```

1.3 阿里命名风格介绍

- 7.【强制】抽象类命名使用Abstract或Base开头；异常类命名使用Exception结尾；测试类命名以它要测试的类的名称开始，以Test结尾。
- 8.【强制】类型与中括号紧挨相连来表示数组。

  正例：定义整形数组 `int[] arrayDemo`。
  反例：在main参数中，使用 `String args[]`来定义。

- 9.【强制】POJO类中的任何布尔类型的变量，都不要加 is 前缀，否则部分框架解析会引起序列化错误。

  反例：定义为基本数据类型 `Boolean isDeleted` 的属性，它的方法也是 `isDeleted()`，框架在反向解析的时候，“误以为”对应的属性名称是 `deleted`，导致属性获取不到，进而抛出异常。

1.3 阿里命名风格介绍

- 10.【强制】包名统一使用小写，点分隔符之间有且仅有一个自然语义的英语单词。包名统一使用单数形式，但是类名如果有复数含义，类名可以使用复数形式。

  正例：应用工具类包名为com.alibaba.ei.kunlun.aap.util、类名为 MessageUtils（此规则参考spring的框架结构）

- 11.【强制】避免在子父类的成员变量之间、或者不同代码块的局部变量之间采用完全相同的命名，使可理解性降低。

  说明：子类、父类成员变量名相同，即使是public类型的变量也能够通过编译，另外，局部变量在同一方法内的不同代码块中同名也是合法的，这些情况都要避免。
  对于非setter/getter 的参数名称也要避免与成员变量名称相同。

1.3 阿里命名风格介绍

反例：

```
public class ConfusingName {
    public int stock;
    protected String alibaba;
    // 非setter/getter的参数名称，不允许与本类成员变量同名
    public void get(String alibaba) {
        if (condition) {
            final int money = 666;
            // ...
        }
        for (int i = 0; i < 10; i++) {
            // 在同一方法体中，不允许与其它代码块中的money命名相同
            final int money = 15978;
            // ...
        }
    }
}
class Son extends ConfusingName {
    // 不允许与父类的成员变量名称相同
    public int stock;
}
```

1.3 阿里命名风格介绍

- 12. 【强制】杜绝完全不规范的缩写，避免望文不知义。

  反例：AbstractClass“缩写”成AbsClass；condition“缩写”成 condi；Function缩写”成Fu，此类随意缩写严重降低了代码的可阅读性。

- 13. 【推荐】为了达到代码自解释的目标，任何自定义编程元素在命名时，使用尽量完整的单词组合来表达。

  正例：对某个对象引用的volatile字段进行原子更新的类名为AtomicReferenceFieldUpdater。 反例：常见的方法内变量为int a;的定义方式。

- 14. 【推荐】在常量与变量的命名时，表示类型的名词放在词尾，以提升辨识度。

  正例：startTime / workQueue / nameList / TERMINATED_THREAD_COUNT
  反例：startedAt / QueueOfWork / listName / COUNT_TERMINATED_THREAD

1.3 阿里命名风格介绍

- 15.【推荐】如果模块、接口、类、方法使用了设计模式，在命名时需体现出具体模式。

  说明：将设计模式体现在名字中，有利于阅读者快速理解架构设计理念。

```
正例：
public class OrderFactory;
public class LoginProxy;
public class ResourceObserver;
```

- 16.【推荐】接口类中的方法和属性不要加任何修饰符号（public 也不要加），保持代码的简洁性，并加上有效的Javadoc注释。尽量不要在接口里定义变量，如果一定要定义变量，确定与接口方法相关，并且是整个应用的基础常量。

```
正例：接口方法签名 void commit();
      接口基础常量 String COMPANY = "alibaba";
反例：接口方法定义 public abstract void f();
```

  说明：JDK8 中接口允许有默认实现，那么这个 default 方法，是对所有实现类都有价值的默认实现。

1.3 阿里命名风格介绍

- 17.接口和实现类的命名有两套规则：

  1）【强制】对于 Service 和 DAO 类，基于 SOA 的理念，暴露出来的服务一定是接口，内部的实现类用 Impl 的后缀与接口区别。

  正例：CacheServiceImpl 实现 CacheService 接口。

  2）【推荐】如果是形容能力的接口名称，取对应的形容词为接口名（通常是–able的形容词）。

  正例：AbstractTranslator实现 Translatable接口。

- 18. 【参考】枚举类名带上 Enum 后缀，枚举成员名称需要全大写，单词间用下划线隔开。

  说明：枚举其实就是特殊的常量类，且构造方法被默认强制是私有。

  正例：枚举名字为ProcessStatusEnum的成员名称：SUCCESS / UNKNOWN_REASON。

1.3 阿里命名风格介绍

- 19.【参考】各层命名规约：

  A) Service/DAO层方法命名规约

  1） 获取单个对象的方法用 get 做前缀。
  2） 获取多个对象的方法用 list 做前缀，复数结尾，如：listObjects。
  3） 获取统计值的方法用 count 做前缀。
  4） 插入的方法用 save/insert 做前缀。
  5） 删除的方法用 remove/delete 做前缀。
  6） 修改的方法用 update 做前缀。

  B) 领域模型命名规约

  1） 数据对象：xxxDO，xxx 即为数据表名。
  2） 数据传输对象：xxxDTO，xxx 为业务领域相关的名称。
  3） 展示对象：xxxVO，xxx 一般为网页名称。
  4） POJO是 DO/DTO/BO/VO 的统称，禁止命名成 xxxPOJO。

2. 常量定义

Definition of Constant Variables

2.1 Java 中对常量的语法规定

- 在Java中，常量（Constants）是一种特殊的变量，其值一旦被初始化后，就不能再被改变。在Java中，常量是通过使用final关键字来定义的。
- 为了使常量更具可读性，通常使用全大写字母，并在单词之间使用下划线（_）作为分隔符。这是Java社区中广泛接受的一种命名约定。

2.2 阿里开发规范中对常量的规定

- 1. 【强制】不允许任何魔法值（即未经预先定义的常量）直接出现在代码中。

  说明：程序员大量的工作其实是在阅读代码，如果出现如下代码

```
if (2 == education) {
  ...
}
```

这样的代码是很难让人理解的，不知道这个 2 代表啥，代表状态还是代表类型?

一个未经定义的字面值会造成很多不可预知的错误。又因为阿里的人员流动非常频繁，没有那么多时间去让新人钻研老人的代码。高效是它永恒的追求，所以才有这条规范。建议定义成如下内容：如果声明一个常量，来替换该魔法值，可能就会使代码的可读性和可维护性大大增加。

```
//大学学历
static final Integer UNIVERSITY_EDUCATION = 2;
```

```
if (UNIVERSITY_EDUCATION  == education) {
    ...
}
```

2.2 阿里开发规范中对常量的规定

- 2. 【强制】在 long 或者 Long 赋值时，数值后使用大写字母 L 标记，不能是小写字母 l，小写容易跟数字混淆，造成误解。

  说明：Long a = 2l; 写的是数字的 21，还是 Long 型的 2 ？

  正例：Long a = 2L;

- 3. 【推荐】不要使用一个常量类维护所有常量，要按常量功能进行归类，分开维护。

  说明：大而全的常量类，杂乱无章，使用查找功能才能定位到修改的常量，不利于理解，也不利于维护。

  正例：缓存相关常量放在类CacheConsts下；系统配置相关常量放在类SystemConfigConsts下。

2.2 阿里开发规范中对常量的规定

- 4.【推荐】常量的复用层次有五层：跨应用共享常量、应用内共享常量、子工程内共享常量、包内共享常量、类内共享常量。

  1） 跨应用共享常量：放置在二方库中，通常是 client.jar 中的 constant 目录下。

  2） 应用内共享常量：放置在一方库中，通常是子模块中的 constant 目录下。

反例：易懂变量也要统一定义成应用内共享常量，两位工程师在两个类中分别定义了“YES”的变量：

类A中：`public static final String YES = "yes";`

类B中：`public static final String YES = "y";`

`A.YES.equals(B.YES)`，预期是`true`，但实际返回为`false`，导致线上问题。

项	说明
一方库	工程内部项目模块依赖的库，一般是本项目或者本工程中的类、方法、接口等。（比如，项目中存在多个模块，模块 A 中引入模块 B 的 jar 包）
二方库	公司内部的，发布到中央仓库 可供其他应用直接依赖、使用的依赖包
三方库	公司外部的开源库，如 spring、apache 依赖等等

2.2 阿里开发规范中对常量的规定

- 4.【推荐】常量的复用层次有五层：跨应用共享常量、应用内共享常量、子工程内共享常量、包内共享常量、类内共享常量。

  3） 子工程内部共享常量：即在当前子工程的 constant 目录下。
  4） 包内共享常量：即在当前包下单独的 constant 目录下。
  5） 类内共享常量：直接在类内部 private static final定义。

2.2 阿里开发规范中对常量的规定

- 5. 【推荐】如果变量值仅在一个固定范围内变化用 enum 类型来定义。

  说明：如果存在名称之外的延伸属性应使用 enum 类型，下面正例中的数字就是延伸信息，表示一年中的第几个季节。

```
public enum SeasonEnum {

    SPRING(1), SUMMER(2), AUTUMN(3), WINTER(4);

    private int seq;

    SeasonEnum(int seq) {
        this.seq = seq;
    }

    public int getSeq() {
        return seq;
    }
}
```

3. 代码风格/代码格式

Code Style

3.1 什么是代码风格

- 代码风格指编程时遵循的格式规范，包括变量命名、缩进、注释等规则，旨在提升代码可读性、维护性和团队协作效率。
- 实际工作中想要实现的目标：在团队、项目中保持风格统一，让代码像同一个人写出来的，整齐划一

3.2 阿里代码格式介绍

- 1.【强制】如果是大括号内为空，则简洁地写成{}即可，大括号中间无需换行和空格；如果是非空代码块则：
  1) 左大括号前不换行。
  2) 左大括号后换行。
  3) 右大括号前换行。
  4) 右大括号后还有else等代码则不换行；表示终止的右大括号后必须换行。

- 2.【强制】左小括号和右边相邻字符之间不出现空格；右小括号和左边相邻字符之间也不出现空格；而左大括号前需要加空格。详见第 5 条下方正例提示。

  反例：`if (空格a == b空格)`

- 3.【强制】if/for/while/switch/do 等 保留字与括号之间 都必须加空格。

3.2 阿里代码格式介绍

- 4. 【强制】任何二目、三目运算符的左右两边都需要加一个空格。

  说明：包括赋值运算符=、逻辑运算符&&、加减乘除符号等。

- 5. 【强制】采用4个空格缩进，禁止使用Tab字符。

  说明：如果使用Tab缩进，必须设置 1 个 Tab 为 4 个空格。IDEA 设置 Tab 为 4 个空格时，请勿勾选 Use tab character；而在 Eclipse 中，必须勾选 insert spaces for tabs。

3.2 阿里代码格式介绍

```
public static void main(String[] args) {
    // 缩进4个空格，禁止使用Tab字符
    String say = "hello";
    // 运算符的左右必须有一个空格
    int flag = 0;
    // 关键词if与括号之间必须有一个空格，括号内的 f与左括号，0与右括号不需要空格
    if (flag == 0) {
        System.out.println(say);
    }
    // 左大括号前加空格且不换行；左大括号后换行
    if (flag == 1) {
        System.out.println("world");
        // 右大括号前换行，右大括号后有else，不用换行
    } else {
        System.out.println("ok");
        // 在右大括号后直接结束，则必须换行
    }
}
```

阿里《开发规范-嵩山版》代码格式 1-5，p5

3.2 阿里代码格式介绍

- 6.【强制】注释的双斜线与注释内容之间有且仅有一个空格。

正例：

```
// 这是示例注释，请注意在双斜线之后有一个空格
String commentString = new String();
```

- 7.【强制】在进行类型强制转换时，右括号与强制转换值之间不需要任何空格隔开。

正例：

```
double first = 3.2d;
int second = (int)first + 2;
```

3.2 阿里代码格式介绍

- 8.【强制】单行字符数限制不超过120个，超出需要换行，换行时遵循如下原则：
  - 1）第二行相对第一行缩进4个空格，从第三行开始，不再继续缩进，参考示例。
  - 2）运算符与下文一起换行。
  - 3）方法调用的点符号与下文一起换行。
  - 4）方法调用中的多个参数需要换行时，在逗号后进行。
  - 5）在括号前不要换行，见反例。

正例：

```
StringBuilder sb = new StringBuilder();
// 超过120个字符的情况下，换行缩进4个空格，并且方法前的点号一起换行
sb.append("yang").append("hao")...
    .append("chen")...
    .append("chen")...
    .append("chen");
```

3.2 阿里代码格式介绍

- 8.【强制】单行字符数限制不超过120个，超出需要换行，换行时遵循如下原则：
  - 4）方法调用中的多个参数需要换行时，在逗号后进行。
  - 5）在括号前不要换行，见反例。

反例：

```
StringBuilder sb = new StringBuilder();
// 超过120个字符的情况下，不要在括号前换行
sb.append("you").append("are")...append
      ("lucky");

// 参数很多的方法调用可能超过120个字符，逗号后才是换行处
method(args1, args2, args3, ...
, argsX);
```

3.2 阿里代码格式介绍

- 9.【强制】方法参数在定义和传入时，多个参数逗号后面必须加空格。

正例：下例中实参的args1，后边必须要有一个空格。

```
method(args1, args2, args3);
```

- 10.【强制】IDE的text file encoding设置为UTF-8；IDE中文件的换行符使用Unix格式，不要使用Windows格式。

解释：
(1) 在Windows中，使用“回车CR('\r')”和“换行LF('\n')”两个字符作为换行符;
(2) Windows系统里面，每行结尾是 回车+换行(CR+LF)，即“\r\n”；
(3) Unix系统里，每行结尾只有 换行LF，即“\n”；

3.2 阿里代码格式介绍

- 11.【推荐】单个方法的总行数不超过80行。
  - 说明：除注释之外的方法签名、左右大括号、方法内代码、空行、回车及任何不可见字符的总行数不超过80行。

正例：代码逻辑分清红花和绿叶，个性和共性，绿叶逻辑单独出来成为额外方法，使主干代码更加清晰；共性逻辑抽取成为共性方法，便于复用和维护。

- 12.【推荐】没有必要增加若干空格来使变量的赋值等号与上一行对应位置的等号对齐。

正例：

```
int one = 1;
long two = 2L;
float three = 3F;
StringBuilder sb = new StringBuilder();
```

  - 说明：增加 sb 这个变量，如果需要对齐，则给 one、two、three 都要增加几个空格，在变量比较多的情况下，是非常累赘的事情。

3.2 阿里代码格式介绍

- 13.【推荐】不同逻辑、不同语义、不同业务的代码之间插入一个空行分隔开来以提升可读性
  - 说明：任何情形，没有必要插入多个空行进行隔开。

4. 注释规约

Comment Specification

4.1 注释-重要性

程序员最讨厌的两件事：一是讨厌自己写注释；二是讨厌别人不写注释；

写注释很烦（作者角度）

理由①—时间压力：项目开发周期紧张，程序员优先完成核心功能开发，牺牲注释编写时间以加速进度

理由②—代码自解释性认知：代码命名和结构足够清晰，无需额外注释。这种认知可能导致忽视注释对维护和团队协作的价值

理由③—维护更新困难：注释需要与代码迭代同步更新，维护成本较高

理由④—潜在误导风险：注释若不及时更新或表述不准确，可能误导后续开发者，反而降低代码可读性

注释很重要（读者角度）

短期记忆让开发者误以为能记住代码逻辑。高估自己未来对代码的理解能力，忽略了团队协作或长期维护的需求。导致技术负债越来越多。

4.1 注释-常见的注释类型

第一种类型，是记录源代码版权和授权，使用一般的星号注释符（/-/），不是本课程重点。

```
/*
 * Copyright (c) 2018, FirstName LastName. All rights reserved.
 */
```

第二种类型，是用来生成用户文档的。使用 Javadoc 要求的格式，文档注释符（/-*/）。 这些文档帮助使用者了解软件的功能和细节，主要面向的是该软件的使用者，称为用户规约（User Specification）。如 Java 的 API 规范的文档。

```
/**
 * A {@code Readable} is a source of characters. Characters from
 * a {@code Readable} are made available to callers of the read
 * method via a {@link java.nio.CharBuffer CharBuffer}.
 *
 * @since 1.5
 */
public interface Readable {
...
}
```

第三种类型，是用来解释源代码的。即：帮助代码的阅读者理解代码。这是大家默认的注释类型，只使用行注释符（//）

4.1 注释-内容

针对第二种注释，是用来生成用户文档。注释内容主要包含三个方面：做什么、为什么、怎么做

```
/**
 * (what) Bean factory to create beans.
 * (why) The class likes Spring IOC framework, but is more lightweight.
 * (how) Create objects from different sources sequentially:
 * user specified object > SPI > configuration > default object.
 */
public class BeansFactory {
    // ...
}
```

类

```
    /**
     * Compute a hailstone sequence.
     *
     * @param n Starting number for sequence.  Assumes n > 0.
     * @return hailstone sequence starting with n and ending with 1.
     */
    public static List<Integer> hailstoneSequence(int n) {
        // 代码略

    }
```

类中的方法

Communicating with other people.

方法的规约 Specification

包括方法签名部分！

Communicating with the computer.

方法的实现体 Implementation

4.1 注释-内容

针对第三种注释，即代码中的注释：注释应该先于代码出现，是代码思路的体现。

```
#循环日期段，从开始日期到结束日期
    #调用选股策略，选出当天合乎交易条件的股票列表
    #循环5分钟K线，从当日的第一个5分钟K线，到最后一个5分钟K线（共12*4=48个）
        #循环每个选股策略选出的股票
            #检查当日K线，或当前5分钟K线，是否会触发买入策略，是则执行买入操作
            #检查当日K线，或当前5分钟K线，是否会触发卖出策略，是则执行卖出操作
        #下一只股票
    #下一个分钟K线
#下一日
```

知乎 @雨后阳光

图片参考：https://www.zhihu.com/question/1916830550880139152/answer/1923688644134298973

4.1 注释-三项原则

- 注释的基本目的是用来提高代码的可读性和可维护性。 不要让注释偏离了这个基本目的。因此给出三项原则：
  - ①准确，错误的注释比没有注释更糟糕。
  - ②必要，多余的注释浪费阅读者的时间。
  - ③清晰，混乱的注释会把代码搞得更乱。

4.2 阿里注释规约介绍

- 1. 【强制】类、类属性、类方法的注释必须使用Javadoc规范，使用/**内容*/格式，不得使用 // xxx方式。
  说明：在IDE编辑窗口中，Javadoc方式会提示相关注释，生成Javadoc可以正确输出相应注释；在IDE中，工程调用方法时，不进入方法即可悬浮提示方法、参数、返回值的意义，提高阅读效率。
- 2. 【强制】所有的抽象方法（包括接口中的方法）必须要用Javadoc注释、除了返回值、参数、异常说明外，还必须指出该方法做什么事情，实现什么功能。
  说明：对子类的实现要求，或者调用注意事项，请一并说明。
- 3. 【强制】所有的类都必须添加创建者和创建日期。
  说明：在设置模板时，注意IDEA的@author为 `${USER}`，而 eclipse 的 @author 为 `${user}`，大小写有区别，而日期的设置统一为 yyyy/MM/dd 的格式。

正例：

```
 /**
* @author yangguanbao
* @date 2016/10/31
*/
```

4.2 阿里注释规约介绍

- 4. 【强制】方法内部单行注释，在被注释语句上方另起一行，使用//注释。方法内部多行注释使用/* */注释，注意与代码对齐。
- 5. 【强制】所有的枚举类型字段必须要有注释，说明每个数据项的用途。
- 6. 【推荐】与其“半吊子”英文来注释，不如用中文注释把问题说清楚。专有名词与关键字保持英文原文即可。

  反例：“TCP连接超时”解释成“传输控制协议连接超时”，理解反而费脑筋。
- 7. 【推荐】代码修改的同时，注释也要进行相应的修改，尤其是参数、返回值、异常、核心逻辑等的修改。
- 8. 【推荐】在类中删除未使用的任何字段、方法、内部类；在方法中删除未使用的任何参数声明与内部变量。
- 9. 【参考】谨慎注释掉代码。在上方详细说明，而不是简单地注释掉。如果无用，则删除。

  说明：代码被注释掉有两种可能性：1）后续会恢复此段代码逻辑。2）永久不用。前者如果没有备注信息，难以知晓注释动机。后者建议直接删掉即可，假如需要查阅历史代码，登录代码仓库即可。

4.2 阿里注释规约介绍

- 10. 【参考】对于注释的要求：第一、能够准确反映设计思想和代码逻辑；第二、能够描述业务含义，使别的程序员能够迅速了解到代码背后的信息。完全没有注释的大段代码对于阅读者形同天书，注释是给自己看的，即使隔很长时间，也能清晰理解当时的思路；注释也是给继任者看的，使其能够快速接替自己的工作。
- 11. 【参考】好的命名、代码结构是自解释的，注释力求精简准确、表达到位。避免出现注释的一个极端：过多过滥的注释，代码的逻辑一旦修改，修改注释又是相当大的负担。

反例：
```
// put elephant into fridge
```

正例
```
put(elephant, fridge);
```

方法名 put，加上两个有意义的变量名 elephant 和 fridge，已经说明了这是在干什么，语义清晰的代码不需要额外的注释。

4.2 阿里注释规约介绍

- 12. 【参考】特殊注释标记，请注明标记人与标记时间。注意及时处理这些标记，通过标记扫描，经常清理此类标记。线上故障有时候就是来源于这些标记处的代码。

  1） 待办事宜（TODO）：

  （标记人，标记时间，[预计处理时间]） 表示需要实现，但目前还未实现的功能。这实际上是一个Javadoc的标签，目前的Javadoc还没 有实现，但已经被广泛使用。只能应用于类，接口和方法（因为它是一个 Javadoc 标签）。

  2） 错误，不能工作（FIXME）：

  （标记人，标记时间，[预计处理时间]） 在注释中用FIXME标记某代码是错误的，而且不能工作，需要及时纠正的情况。

4.2 阿里注释规约介绍

- 其他编码规约不一一列举。请仔细阅读《阿里巴巴 Java 开发手册 V1.7.0 》，具体包括
  - OOP规约
  - 日期时间
  - 集合处理
  - 并发处理
  - 控制语句
  - 前后端规约
  - 其他
- 除了编码规约之外，还有：异常日志；单元测试；安全规约；MySQL数据库；工程结构以及设计规约。
- 这些规约都是阿里巴巴多年的沉淀，经过时间考验的成型方案，要求仔细研究。

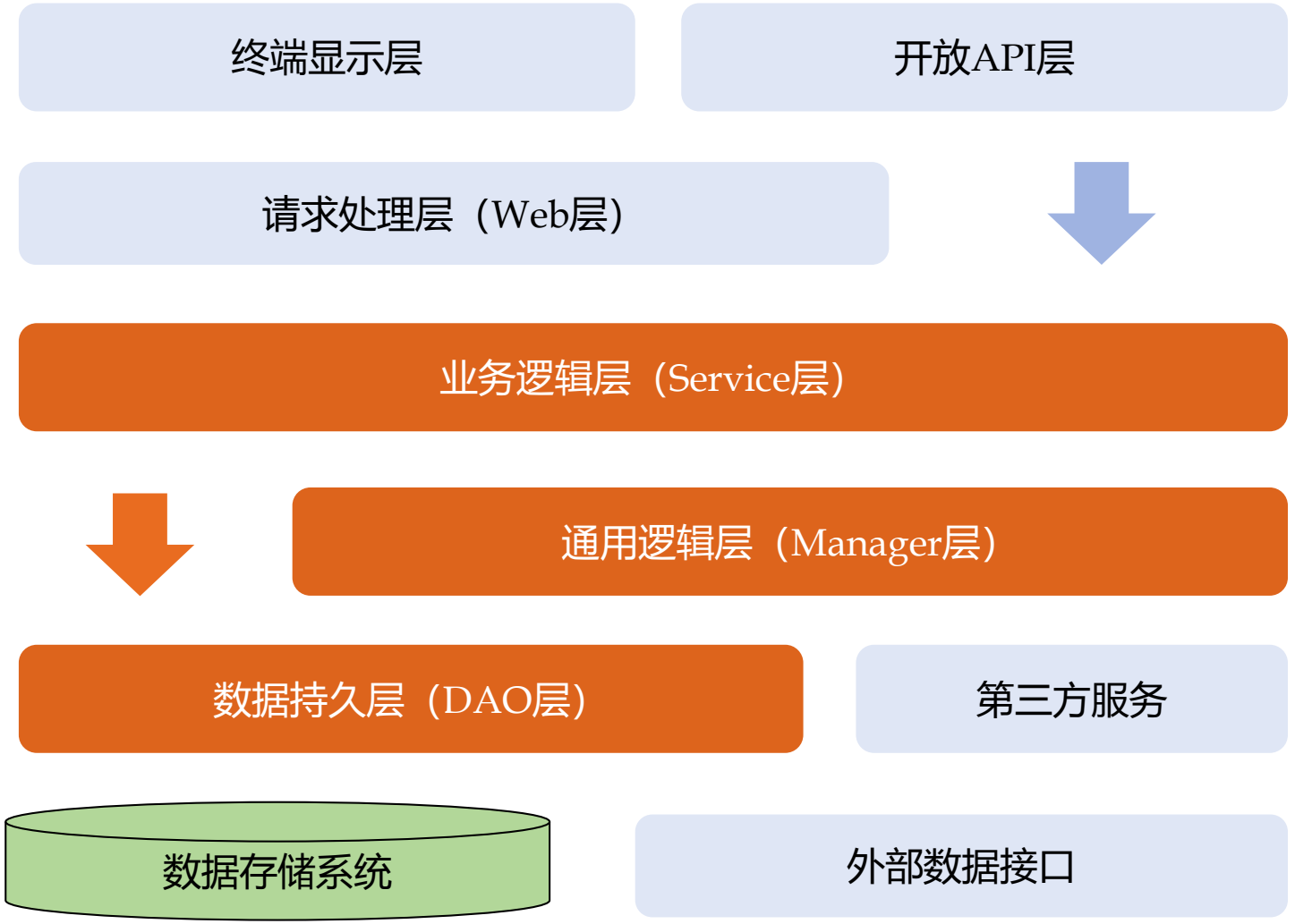
5. 工程结构
Project Structure

5.1 阿里巴巴应用分层

1. 【推荐】根据业务架构实践，结合业界分层规范与流行技术框架分析，推荐分层结构如图所示。

默认上层依赖于下层，箭头关系表示可直接依赖。

如：开放API层可以依赖于Web层（Controller层），也可以直接依赖于Service层，依此类推：



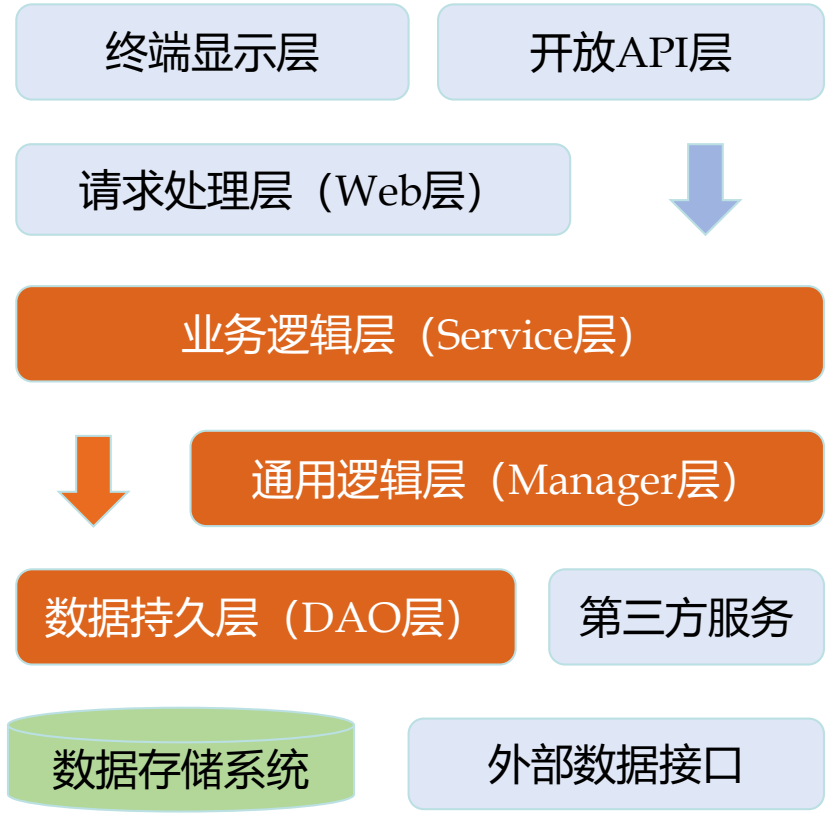
终端显示层：各个端的模板渲染并执行显示的层。当前主要是 velocity 渲染，JS 渲染，JSP 渲染，移动端展示等。

开放 API 层：可直接封装 Service 接口暴露成 RPC 接口；通过 Web 封装成 http 接口；网关控制层等。

Service层：相对具体的业务逻辑服务层。

DAO层：数据访问层，与底层MySQL、Oracle、Hbase、OB等进行数据交互。

第三方服务：包括其它部门RPC服务接口，基础平台，其它公司的HTTP接口，如淘宝开放平台、支付宝付款服务、高德地图服务等。



Web层：主要是对访问控制进行转发，各类基本参数校验，或者不复用的业务简单处理等。

Manager层：通用业务处理层，它有如下特征：

1） 对第三方平台封装的层，预处理返回结果及转化异常信息，适配上层接口。

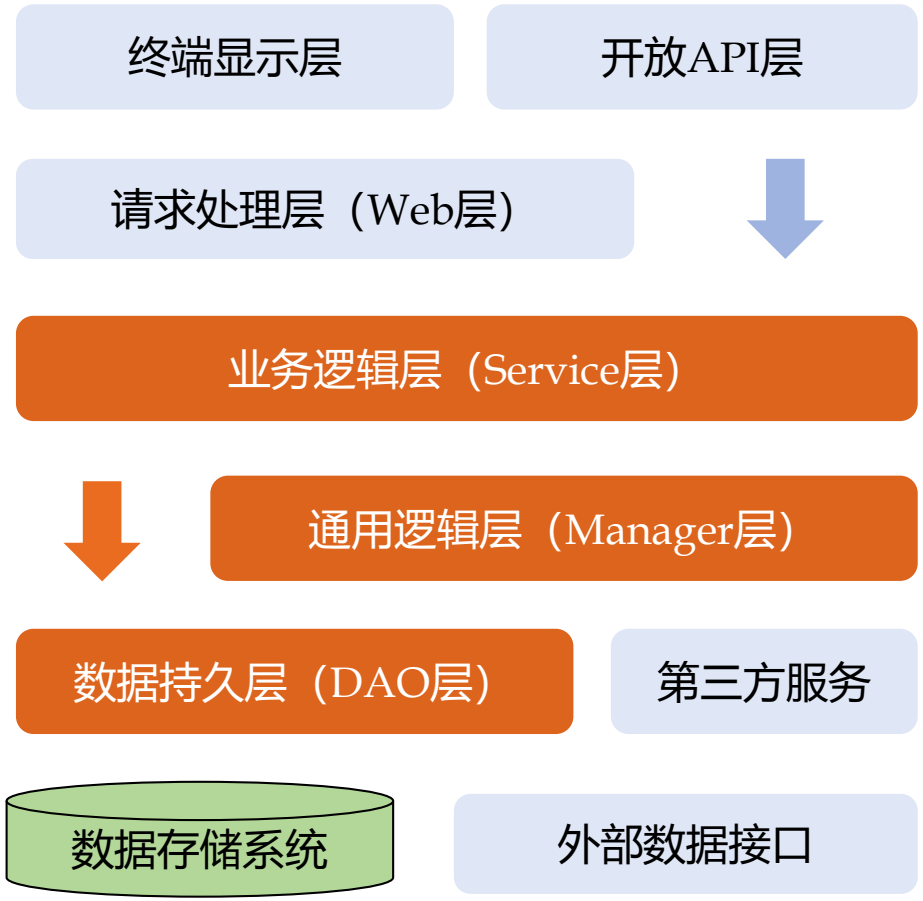
2） 对Service层通用能力的下沉，如缓存方案、中间件通用处理。

3） 与DAO层交互，对多个DAO的组合复用。

外部数据接口：外部（应用）数据存储服务提供的接口，多见于数据迁移场景中。

5.2 分层异常处理规约

- 2.【参考】（分层异常处理规约）



- 开放接口层要将异常处理成错误码和错误信息方式返回。
- Web层绝不应该继续往上抛异常，因为已经处于顶层，如果意识到这个异常将导致页面无法正常渲染，那么就应该直接跳转到友好错误页面，尽量加上友好的错误提示信息。
- 在Service层出现异常时，必须记录出错日志到磁盘，尽可能带上参数信息，相当于保护案发现场
- Manager 层与 Service 同机部署，日志方式与 DAO 层处理一致；如果是单独部署，则采用与 Service层 一致的处理方式
- 在DAO层，产生的异常类型有很多，无法用细粒度的异常进行 catch，可以使用 catch(Exception e) 方式，并 throw new DAOException(e)，不需要打印日志，因为日志在Manager/Service层一定需要捕获并打印到日志文件中去，如果同台服务器再打日志，浪费性能和存储。

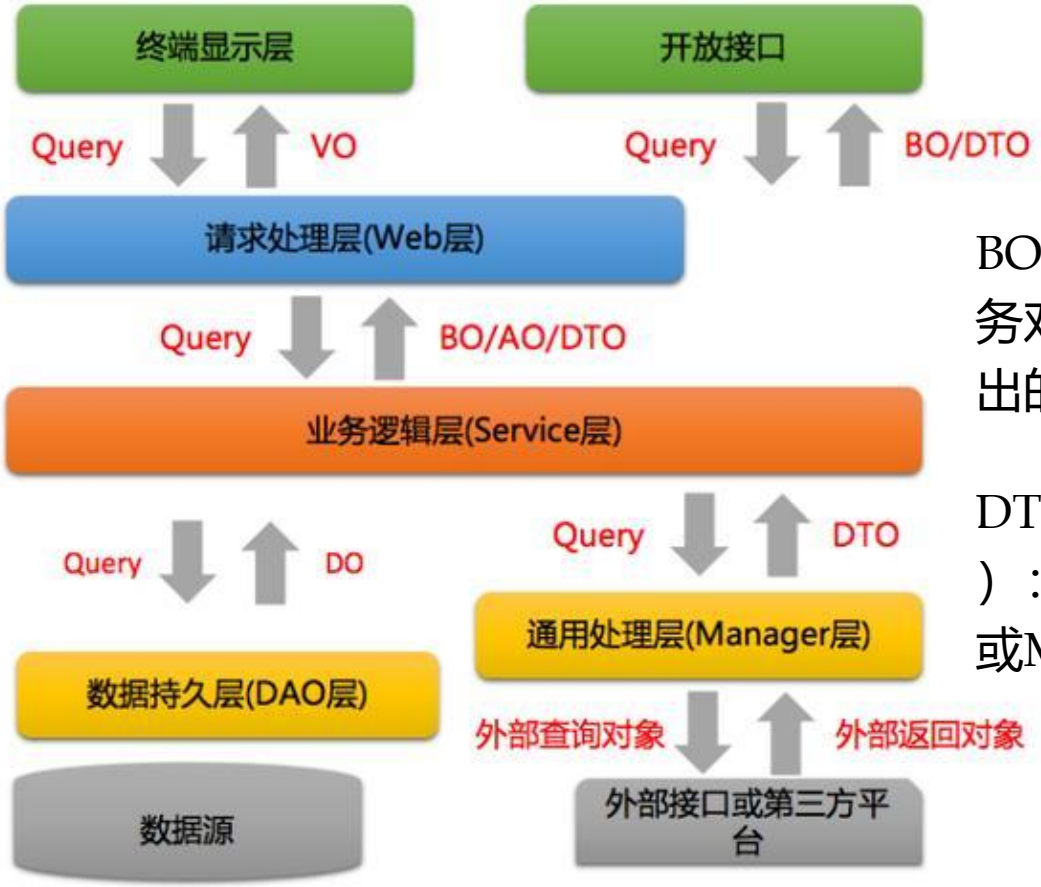
5.3 分层领域模型规约

- 3. 【参考】分层领域模型规约：

Query：数据查询对象，各层接收上层的查询请求。注意超过2个参数的查询封装，禁止使用Map类来传输。

VO（View Object）：显示层对象，通常是Web向模板渲染引擎层传输的对象。

DO（Data Object）：此对象与数据库表结构一一对应，通过DAO层向上传输数据源对象



BO（Business Object）：业务对象，可以由Service层输出的封装业务逻辑的对象。

DTO（Data Transfer Object）：数据传输对象，Service或Manager向外传输的对象

6. 提高代码可读性的几个技巧

Several tips for improving code readability

6.1 把代码分割成更小的单元块

大部分人阅读代码的习惯都是：先看整体再看细节。
所以，我们要有模块化和抽象思维，善于将大块的复杂逻辑提炼成类或者函数，屏蔽掉细节，让阅读代码的人不至于迷失在细节中，这样能极大地提高代码的可读性。

示例：请阅读以下代码

```
// 重构前的代码
public void invest(long userId, long financialProductId) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTime(date);
    calendar.set(Calendar.DATE, (calendar.get(Calendar.DATE) + 1));
    if (calendar.get(Calendar.DAY_OF_MONTH) == 1) {
        return;
    }
    //...
}
```

6.1 把代码分割成更小的单元块

```
// 重构后的代码：提炼函数之后逻辑更加清晰
public void invest(long userId, long financialProductId) {
    if (isLastDayOfMonth(new Date())) {
        return;
    }
    //...
}
```

将重构前的逻辑封装成isLastDayOfMonth()

```
public boolean isLastDayOfMonth(Date date) {
    Calendar calendar = Calendar.getInstance();
    calendar.setTime(date);
    calendar.set(Calendar.DATE, (calendar.get(Calendar.DATE) + 1));
    if (calendar.get(Calendar.DAY_OF_MONTH) == 1) {
        return true;
    }
    return false;
}
```

重构后，发现：将复杂的逻辑代码提炼成函数，大大提高了代码的可读性。

6.2 避免函数参数过多

- 针对参数过多的情况，一般有 3 种处理方法。

方法一：考虑函数是否职责单一，是否能通过拆分成多个函数的方式来减少参数。示例代码如下所示：

```
public void getUser(String username, String telephone, String email);
```



```
// 拆分成多个函数
public void getUserByUsername(String username);
public void getUserByTelephone(String telephone);
public void getUserByEmail(String email);
```

6.2 避免函数参数过多

- 针对参数过多的情况，一般有 3 种处理方法。

方法二：将函数的参数封装成对象。示例代码如下所示：

```
public void postBlog(String title, String summary, String keywords, String content, long
authorId)
```

↓

```
// 将参数封装成对象
public class Blog {
    private String title;
    private String summary;
    private String keywords;
    private Strint content;
    private String category;
    private long authorId;
}
```

↓

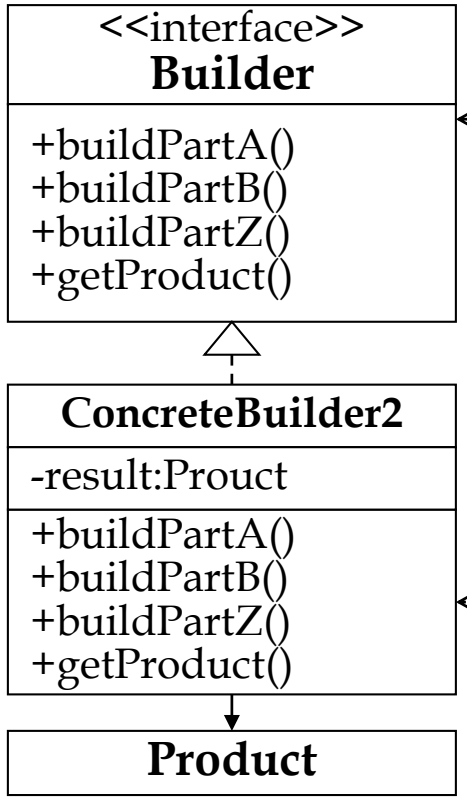
```
public void postBlog(Blog blog);  // 入参变成封装后的对象
```

6.2 避免函数参数过多

- 针对参数过多的情况，一般有 3 种处理方法。

方法三：建造者模式

建造者/生成器（Builder）接口声明在所有类型生成器中通用的产品构造步骤。



主管（Director）类定义调用构造步骤的顺序，这样你就可以创建和复用特定的产品配置。

具体生成器（Concrete Builders）提供构造过程的不同实现。既然是子类，显然可以增加不同的建造器

客户端（Client）需要知道 **Director** 和具体的建造者

6.2 避免函数参数过多

- 在《创建型模式》一章中，练习3：电脑主机的主要部件包括cpu，主板，内存，硬盘等。不同的电脑发烧友对部件的要求不一样，如何根据不同的元器件，产生不同的电脑主机对象？







6.2 避免函数参数过多

```
public class PC {
    private String cpu;
    private String memory;
    private String mainboard;
    private String power;
    private String disk;
    private String graphics;

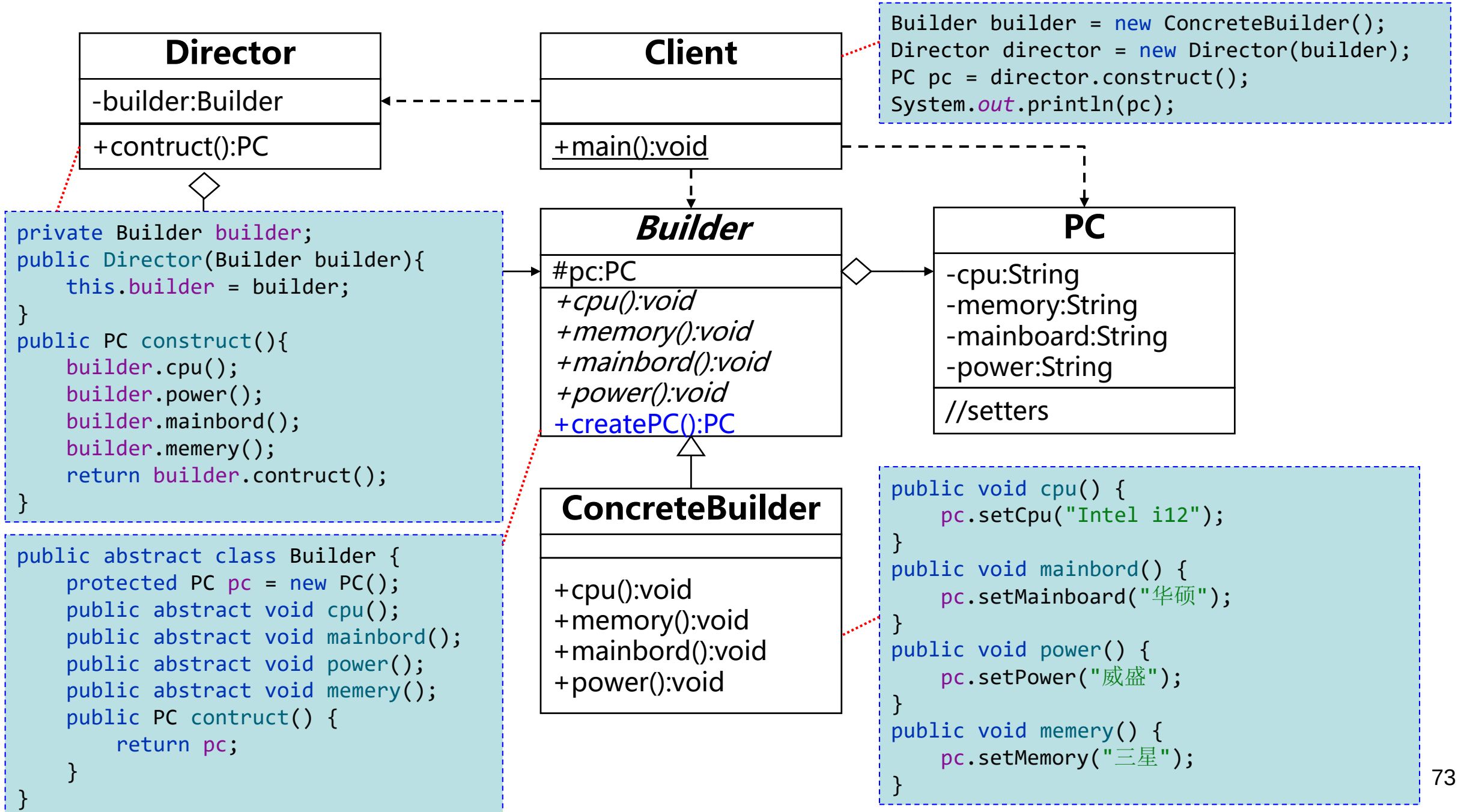
    public PC(String cpu, String memory, String mainboard,
                          String power, String disk, String graphics) {
        this.cpu = cpu;
        this.memory = memory;
        this.mainboard = mainboard;
        this.power = power;
        this.disk = disk;
        this.graphics = graphics;
    }
    // setter和getter略...
}
```

6.2 避免函数参数过多

```
public class Client {
    public static void main(String[] args) {
        //构建CompatiblePC对象
        PC pc = new PC("intel i11",
                       "三星32G",
                       "华硕",
                       "华硕",
                       "西部数据",
                       "Geforece3900");

        System.out.println(pc);
    }
}
```

代码可读性会非常差，而且很容易引入错误。


Director
-builder:Builder
+contruct():PC
Client
+main():void
Builder builder = new ConcreteBuilder();
Director director = new Director(builder);
PC pc = director.construct();
System.out.println(pc);
Builder
#pc:PC
+cpu():void
+memory():void
+mainbord():void
+power():void
+createPC():PC
PC
-cpu:String
-memory:String
-mainboard:String
-power:String
//setters
ConcreteBuilder
+cpu():void
+memory():void
+mainbord():void
+power():void
private Builder builder;
public Director(Builder builder){
this.builder = builder;
}
public PC construct(){
builder.cpu();
builder.power();
builder.mainbord();
builder.memery();
return builder.contruct();
}
public abstract class Builder {
protected PC pc = new PC();
public abstract void cpu();
public abstract void mainbord();
public abstract void power();
public abstract void memery();
public PC contruct() {
return pc;
}
}
public void cpu() {
pc.setCpu("Intel i12");
}
public void mainbord() {
pc.setMainboard("华硕");
}
public void power() {
pc.setPower("威盛");
}
public void memery() {
pc.setMemory("三星");
}

6.2 避免函数参数过多

- 开发中也可以将 Builder 和 Director 合并，并做成 PC 的内部类

```
public class PC {
    private String cpu;
    private String screen;
    private String memory;
    private String mainboard;
    private PC(Builder builder) {
        cpu = builder.cpu;
        screen = builder.screen;
        memory = builder.memory;
        mainboard = builder.mainboard;
    }
    public static final class Builder {
        private String cpu;
        private String screen;
        private String memory;
        private String mainboard;
        public Builder() {}
        // 接左侧代码
```

```
  public Builder cpu(String val) {
            cpu = val;
            return this;
   }
   public Builder screen(String val) {
        screen = val;
        return this;
    }
    public Builder memory(String val) {
        memory = val;
        return this;
    }
    public Builder mainboard(String val) {
        mainboard = val;
        return this;
    }
    public PC build() {
        return new PC(this);}
}
```

6.2 避免函数参数过多

```
// 优化前
public class Client {
    public static void main(String[] args) {
        //构建PC对象
         PC pc = new PC("intel i11", "三星32G", "华硕", "华硕", "西部数据","Geforece3900");
       System.out.println(pc);
    }
}
```

Stink

```
// 优化后
public class Client {
    public static void main(String[] args) {
        PC pc = new PC.Builder()
                .cpu("intel")
                .mainboard("华硕")
                .memory("金士顿")
                .screen("三星")
                .build();
        System.out.println(pc);
    }
}
```

Good

对比优化前，优化后的代码明显具有灵活性

6.3 勿用函数参数来控制逻辑

- 不要在函数中使用布尔类型的标识参数来控制内部逻辑，true 的时候走这块逻辑，false 的时候走另一块逻辑。这明显违背了单一职责原则和接口隔离原则。建议将其拆成两个函数，可读性上也要更好

```
public void buyCourse(long userId, long courseId, boolean isVip);
```



```
// 将其拆分成两个函数
public void buyCourse(long userId, long courseId);
public void buyCourseForVip(long userId, long courseId);
```

6.4 函数设计要职责单一

- 对于函数的设计来说，更要满足单一职责原则

```
public boolean checkUserIfExisting(String telephone, String username, String email) {
    if (!StringUtils.isBlank(telephone)) {
        User user = userRepo.selectUserByTelephone(telephone);
        return user != null;
    }
    if (!StringUtils.isBlank(username)) {
        User user = userRepo.selectUserByUsername(username);
        return user != null;
    }
    if (!StringUtils.isBlank(email)) {
        User user = userRepo.selectUserByEmail(email);
        return user != null;
    }
    return false;
}
```

```
// 拆分成三个函数
public boolean checkUserIfExistingByTelephone(String telephone);
public boolean checkUserIfExistingByUsername(String username);
public boolean checkUserIfExistingByEmail(String email);
```

6.5 移除过深的嵌套层次

- 代码嵌套层次过深往往是因为 if-else、switch-case、for 循环过度嵌套导致的，嵌套最好不超过两层。过深的嵌套本身理解起来就比较费劲，容易因为代码多次缩进，导致嵌套内部的语句超过一行的长度而折成两行，影响代码的整洁

6.5 移除过深的嵌套层次

```
// 例如：示例一
public double caculateTotalAmount(List<Order> orders) {
    if (orders == null || orders.isEmpty()) {
        return 0.0;
    } else { // 此处的 else 可以去掉
        double amount = 0.0;
        for (Order order : orders) {
            if (order != null) {
                amount += (order.getCount() * order.getPrice());
            }
        }
        return amount;
    }
}
```

6.5 移除过深的嵌套层次

```
// 示例二
public List<String> matchStrings(List<String> strList, String substr) {
    List<String> matchedStrings = new ArrayList<>();
    if (strList != null && substr != null) {
        for (String str : strList) {
            if (str != null) { // 跟下面的if语句可以合并在一起
                if (str.contains(substr)) {
                    matchedStrings.add(str);
                }
            }
        }
    }
    return matchedStrings;
}
```

6.5 移除过深的嵌套层次

- 使用编程语言提供的 continue、break、return 关键字，提前退出嵌套。代码示例如下所示：

```
// 重构前的代码
public List<String> matchStrings(List<String> strList, String substr) {
    List<String> matchedStrings = new ArrayList<>();
    if (strList != null && substr != null) {
        for (String str : strList) {
            if (str != null && str.contains(substr)) {
                matchedStrings.add(str);
                // 此处还有10行代码...
            }
        }
    }
    return matchedStrings;
}
```

6.5 移除过深的嵌套层次

```
// 重构后的代码：使用continue提前退出
public List<String> matchStrings(List<String> strList,String substr) {
    List<String> matchedStrings = new ArrayList<>();
    if (strList != null && substr != null){
        for (String str : strList) {
            if (str == null || !str.contains(substr)) {
                continue;
            }
            matchedStrings.add(str);
            // 此处还有10行代码...
        }
    }
    return matchedStrings;
}
```

6.5 移除过深的嵌套层次

- 调整执行顺序来减少嵌套。具体的代码示例如下所示：

```
// 重构前的代码
public List<String> matchStrings(List<String> strList,String substr) {
    List<String> matchedStrings = new ArrayList<>();
    if (strList != null && substr != null) {
        for (String str : strList) {
            if (str != null) {
                if (str.contains(substr)) {
                    matchedStrings.add(str);
                }
            }
        }
    }
    return matchedStrings;
}
```

6.5 移除过深的嵌套层次

- 调整执行顺序来减少嵌套。具体的代码示例如下所示：

```
// 重构后的代码：先执行判空逻辑，再执行正常逻辑

public List<String> matchStrings(List<String> strList, String substr) {
    if (strList == null || substr == null) { //先判空
         return Collections.emptyList();
    }
    List<String> matchedStrings = new ArrayList<>();
    for (String str : strList) {
        if (str != null) {
            if (str.contains(substr)) {
                matchedStrings.add(str);
            }
        }
    }
    return matchedStrings;
}
```

6.5 移除过深的嵌套层次

- 将部分嵌套逻辑封装成函数调用，以此来减少嵌套。具体的代码示例如下所示：

```
// 重构前的代码
public List<String> appendSalts(List<String> passwords) {
    if (passwords == null || passwords.isEmpty()) {
        return Collections.emptyList();
    }
    List<String> passwordsWithSalt = new ArrayList<>();
    for (String password : passwords) {
        if (password == null) {
            continue;
        }
        if (password.length() < 8) {
            // ...
        } else {
            // ...
        }
    }
    return passwordsWithSalt;
}
```

6.5 移除过深的嵌套层次

```
// 重构后的代码：将部分逻辑抽成函数
public List<String> appendSalts(List<String> passwords) {
    if (passwords == null || passwords.isEmpty()) {
        return Collections.emptyList();
    }
    List<String> passwordsWithSalt = new ArrayList<>();
    for (String password : passwords) {
        if (password == null) {
            continue;
        }
        passwordsWithSalt.add(appendSalt(password));
    }
    return passwordsWithSalt;
}
```

6.5 移除过深的嵌套层次

```
private String appendSalt(String password) {
    String passwordWithSalt = password;
    if (password.length() < 8) {
        // ...
    } else {
        // ...
    }
    return passwordWithSalt;
}
```

除此之外，常用的还有通过使用多态来替代 if-else、switch-case 条件判断的方法

6.5 学会使用解释性变量

- 常用的用解释性变量来提高代码的可读性。

重构前

```
public double CalculateCircularArea(double radius) {
    return (3.1415) * radius * radius;
}
```

重构后

```
public static final Double PI = 3.1415;

public double CalculateCircularArea(double radius) {
    return PI * radius * radius;
}
```

6.5 学会使用解释性变量

```
if (date.after(SUMMER_START) && date.before(SUMMER_END)) {
    // ...
} else {
    // ...
}
```

引入解释性变量后逻辑更加清晰

```
boolean isSummer = date.after(SUMMER_START) && date.before(SUMMER_END);
if(isSummer){
    // ...
} else {
    // ...
}
```

本章小结

- 设计原则、设计模式可以有效地改善代码质量，但这些知识的合理应用非常依赖于个人经验，用不好适得其反。
- 而开发规范正好相反，在代码细节方面，能够立竿见影地改善代码质量。
- 开发规范包括编码规范、异常日志规范、单元测试规范、安全规约、MySQL数据库规范、工程结构和设计规约等。
- 重点掌握阿里巴巴的开发手册（嵩山版 1.7.1）

The end

December 12, 2025